

Configure the Agent, Not the Model

Study guide

Created by Swarup Biswas · swarupbiswas.com

This guide is your reference after the session. It covers everything the session taught, in reading order, with the evidence behind each claim and a take-home programme at the end. Keep it next to the four-round walkthrough (your step-by-step lab record) and the gap menu (your skill templates): the walkthrough is for doing, this guide is for understanding and reviewing.

1. The core idea

You configured a **harness**, not a model. Agentic tools do not expose model settings such as temperature or fine-tuning; they expose files that decide what the model is given at the start of every conversation. The same model answers like a stranger in an unconfigured folder and like a colleague in a configured one — and because the files describe your work rather than the AI, they survive model upgrades and even a change of vendor.

You proved this in the four-round experiment: one folder of ordinary work documents, one question asked four times, one file added between rounds. The model never changed; the answers did.

Round	You added	What changed
1	Nothing	Tidy formatting, missing knowledge — initials unexpanded, vague dates, mixed currency styles
2	<code>AGENTS.md</code>	Real names, real dates, the house style
3	A rule	Open questions listed before action items, in every response
4	A skill	The full minutes structure, one owner per action item, and an honest "decisions made: none"

2. The four layers

Two questions separate the layers: **when does it load**, and **who decides**.

Layer	When it loads	Who decides	File
Context	Every conversation	Nobody — always on	<code>AGENTS.md</code>
Rules	Every conversation	Nobody — always on	<code>.agents/rules/</code>
Skills	When the task matches	The agent	<code>.agents/skills/</code>
Commands (workflows)	When you invoke them	You	<code>.agents/workflows/</code>

The decision framework. Does the agent need this in every conversation? If it is project fact, it goes in `AGENTS.md`; if it is a hard constraint, it is a rule. Needed only for certain tasks? A skill. A sequence

you trigger yourself? A workflow. None of those? Just say it in the chat — not everything deserves to be codified, and the research (section 4) shows over-codification has a real cost.

3. Where the files go

Scope	Context	Skills
This project — travels with the folder	<code>AGENTS.md</code> at the folder root	<code>.agents/skills/<name>/SKILL.md</code>
Global, Antigravity — follows you	<code>~/.gemini/AGENTS.md</code>	<code>~/.gemini/config/skills/</code>
Global, Zed — follows you	<code>~/.config/zed/AGENTS.md</code>	<code>~/.agents/skills/</code>

The placement rule: **team truths go in the project; personal compensations go global.** A shared folder's AGENTS.md belongs to the team — your personal gap lines would pollute it. Your gap skill is yours: installed globally, it follows you to every project. Rules and workflows follow the same pattern in folders next to `skills/` inside `.agents/`.

Two operational rules that prevent most failures:

1. **New file, then new chat.** Configuration is read when a conversation starts. A chat that was already open cannot see a file you added afterwards — it will look as though your file did nothing.
2. **Use the canonical paths.** A misplaced file sometimes still "works" because the agent reads workspace files as content, but only the standard paths make your configuration portable to other tools.

4. What the evidence says

Every claim in the session traces to a named study. The four worth remembering:

- **Context files pay only when specific.** A field study across 124 pull requests found a well-written AGENTS.md made agents 28.6% faster at the median (arXiv:2601.20404); a separate evaluation found context files raised inference cost 20% with no success gain when they carried generic project prose (arXiv:2602.11988). Resolution: explicit instructions are followed and pay for themselves; overview essays cost tokens and change nothing. Write working agreements, not descriptions.
- **Human-plus-AI wins only when engineered.** Across 106 experiments, the average human-AI team performed *worse* than the better of the two alone (Vaccaro, Almaatouq & Malone, Nature Human Behaviour 2024). Teams win when the two sides are strong at different things and the human knows which is which — which is why naming your own gap is the mechanism, not a warm-up.
- **AI helps most where your skill is lowest — and can erode what you still need.** A ~4,900-developer field experiment found 26% more completed tasks, with the largest gains for the least experienced (Cui et al., 2024). But professionals who leaned on AI assistance for months measurably lost unassisted skill (Budzyń et al., Lancet Gastro & Hep 2025). Hence the two modes: **compensate** (the agent does what you never need to own) versus **coach** (the agent teaches what you still want to learn, by asking rather than answering).
- **Feelings are not a verification method.** In a randomised trial, experienced developers using AI on their own projects took 19% *longer* — while believing they had been 20% faster (METR, arXiv:2507.09089). Your evidence is a saved before-and-after pair, never an impression. Re-run the contrast check monthly.

5. What a skill actually is

The model has read most of the public web; what it has never read is how your team works. That missing piece is **procedural knowledge** — how to do the work, in what order, with what judgement — and no amount of general intelligence supplies it, because it was never written down anywhere the model could read. A skill is that write-down.

Four ways to give an agent knowledge, and where skills sit:

Method	What it gives the agent	Shape
MCP	Tool access — what it can reach, not when to use it	Live connections to services
RAG	Factual knowledge pulled at run time	Retrieved chunks from a knowledge base
Fine-tuning	Knowledge baked into the model's weights	Permanent, expensive retraining
Skills	Procedural knowledge — how, in what order, with what judgement	Markdown under version control

These complement rather than compete: a skill can tell the agent when to call an MCP tool and what to do with what RAG retrieves. The memory analogy: RAG is semantic memory (knowing facts), chat logs are episodic memory (remembering what happened), and skills are procedural memory (knowing how)

— like riding a bicycle, written down for your agent. (Taxonomy: Xu & Yan, arXiv:2602.12430; memory framework: Sumers et al., arXiv:2309.02427.)

6. The anatomy of a skill

A skill is a folder with one required file, `SKILL.md` :

- **Frontmatter** (between the `---` lines): a `name` matching the folder, and a `description`. The description is a **router, not a label** — the agent matches the words you type against it, so write it like search keywords: what the skill does, then the phrases a person actually uses, including the casual ones.
- **Body**: numbered steps, then the guardrails — a **NEVER list** (prohibitions hold better than suggestions), **exit criteria** (observable yes-or-no checks that define done), and **anti-rationalisation lines** (pre-written counters to "this case is different"). Build guardrails from shortcuts you actually catch the agent taking.
- **Progressive disclosure** keeps a large library affordable: at startup the agent reads only each skill's name and description (~100 tokens); the body loads on a match; `references/` files load at the step that needs them; scripts run without their code ever entering the conversation. Thirty skills cost about 3,000 tokens of awareness, not 150,000 of content. Practical rule: keep the body lean; push templates and long detail into `references/`.

Trigger tuning, when a skill does not fire:

Symptom	Fix
Generic answer, no skill	Add the exact words you typed to the description
Fires on the wrong requests	Add exclusions: "do not use for..."
Fires but output drifts	The description is fine — fix the body: steps, NEVER list, exit criteria

One tuning round is normal. Trigger problems live in the description; behaviour problems live in the body; and every edit needs a fresh conversation.

7. Deploying, and staying safe

The three-part deploy check. Deployed means three checkable facts: the behaviour delta (your saved before-and-after files differ, visibly), the first-ask trigger (the skill fires on natural phrasing after at most one tuning round), and the durable install (the files live in the shared project folder, where the team inherits them, or your global folder, where they follow you).

Portability — what travels where (verified July 2026):

Tool	AGENTS.md	<code>.agents/skills/</code>
Antigravity	Native	Native
Zed	In a fallback chain	Native
Gemini CLI	Via one setting	Native
Claude Code	Bridge file needed	Copy to <code>.claude/skills/</code>
Cursor / Copilot / Codex	Native	Not documented

The skill folder travels better than the context file because a skill carries its own trigger. The honest edges: Claude Code needs a one-line bridge file (a `CLAUDE.md` containing `@AGENTS.md`), and in Zed a

leftover `.cursorrules` file silently wins over AGENTS.md.

Security. A skill is instructions your agent will follow and scripts it may run — installing one is an act of trust, and the risk is documented: in the first marketplace audit (February 2026), 341 of 2,857 skills were malicious, 824 two weeks later, delivering credential-stealing malware through a "paste these commands" section (Koi Security). The named threats are prompt injection in the instructions, tool poisoning through self-granted permissions, and malware in `scripts/`. The habits: install only from sources you trust, read the SKILL.md **and its scripts** before enabling, prefer version-controlled team copies over live marketplace installs, and treat trust prompts (like Zed's worktree trust) as the defence they are, not as friction.

8. Your take-home programme

1. **Re-run the four rounds** from the walkthrough on a folder of your own documents — the pattern works anywhere.
2. **Add your gap's AGENTS.md line** from the gap menu to your own working folder, and verify it with a before-and-after pair.
3. **Build your second skill** from the gap menu (or its coding edition if you code daily), with the same loop: describe, test, tune, verify.
4. **Run the self-check quiz** (six which-layer scenarios, answers included) to confirm the decision framework stuck.
5. **Monthly:** re-run the before-and-after contrast check on your configuration, and review your skills the way you review any team document — they are part of the work product now.

9. Glossary

Term	Meaning
Agent	The AI system in your editor that reads files, runs tools, and answers in chat
Harness	The machinery around the model that decides what each conversation is given — what you actually configure
AGENTS.md	The open-standard context file read at the start of every conversation
Rule	An always-on constraint the agent must follow in every response
Skill	A folder with a SKILL.md carrying procedural knowledge, loaded when the task matches its description
Workflow / command	A saved sequence you trigger explicitly by name
Description (of a skill)	The router: the text the agent matches your words against to decide whether to load the skill
Progressive disclosure	Loading skill content in stages — name and description first, body on match, references and scripts on demand
Frontmatter	The metadata block between <code>---</code> lines at the top of SKILL.md
NEVER list / exit criteria	A skill's guardrails: explicit prohibitions, and observable checks that define "done"
MCP	Model Context Protocol — gives the agent live tool connections (what it can reach)
RAG	Retrieval-augmented generation — supplies facts pulled from a knowledge base at run time
Fine-tuning	Retraining that bakes knowledge into the model's weights
Worktree trust	An editor's gate that keeps a received folder's skills switched off until you approve them

10. Sources

The claims in this guide trace to: agents.md and the Agentic AI Foundation (the AGENTS.md standard); arXiv:2601.20404 and arXiv:2602.11988 (context-file evidence); Vaccaro, Almaatouq & Malone, *Nature Human Behaviour* 2024 (human-AI complementarity); Cui et al. 2024 and Budzyń et al., *Lancet Gastroenterology & Hepatology* 2025 (skill-differential gains and deskilling); METR, arXiv:2507.09089 (perceived versus measured benefit); Xu & Yan, arXiv:2602.12430 (the skills taxonomy and vulnerability base rates); Sumers et al., arXiv:2309.02427 (the memory framework); Anthropic engineering, October 2025 (Agent Skills); Koi Security, February 2026 (the marketplace incident); and vendor documentation for AntigraVity, Zed, Gemini CLI, and Claude Code, accessed 11 July 2026. Tool paths and quotas change quarterly — when this guide disagrees with your tool's current documentation, trust the documentation and tell your instructor.